9 Classes and Objects 2

Viope instruction: Exercises with multiple source files

1. Copy/paste the source code of your program class to the Viope code editor window.
   * If there is a **package** directive, then remove it in Viope.

2. Copy/paste the other class definition to the Viope code editor window below the program class code.
   * Remove the **public** keyword from the class definition. Viope allows only the program class to be public.

3. If you have any ***import*** directives in your source texts, you have to move them to the top of the Viope editor window.
   * Place them before the program class code.

IMPORTANT!

- ✓ Please study the course materials carefully ***before asking for advice***. Thanks!
- ✓ The main objectives of this set of simplified exercises are to *familiarize yourself to object-oriented programming ideas* and to *strengthen your basic skills in creating program logic*. Therefore, it is important that you (1) ***read the exercise texts carefully*** and (2) ***follow carefully the instruction given in the exercise texts***.

Objects City Names

Write a program called **ObjectsCityNames** that first inputs city names from the user until the user decides to quit by entering "quit". Then, the program should print the city names in uppercase in alphabetical order.

* Use the sort method of the Collections class.

Example output:

```
Enter city name: London
Enter city name: Amsterdam
Enter city name: Stockholm
Enter city name: Paris
Enter city name: quit

AMSTERDAM
LONDON
PARIS
STOCKHOLM
```

NOTE: In Java, strings are objects. In this exercise, you do **not** need to create your own class for objects.

Objects Song ArrayList

Write a program called **ObjectsSongArrayList**. The program is a variation of the ***ObjectsSongArray*** program.

Use the **ArrayList** class and create a list of *Songs*. Reuse the **Song** class from the previous exercise.

Create three *Song* objects and save their references to the list. See the example output for the song names, artists and years. Finally, the program should iterate the list and print a song list exactly as shown in the example output.

* Hint: To make it easier, you can use the *enhanced for loop* to iterate the list.

Example output:

```
=== List of songs ===
Adele: Easy on Me (2021)
Ed Sheeran: Shivers (2021)
Larkin Poe: Holy Ghost Fire (2020)
```

Objects Translator ArrayList

Write a program called **ObjectsTranslatorArrayList** that first inputs pairs of English and Finnish words from the user until the user enters "ok" as the English word.

Next, the program should input English words from the user until the user decides to quit by entering "quit". For each entered word, the program prints the corresponding Finnish word. If the word is unknown, the program should display "**Unknown word**".

NOTE! Reuse the **WordPair** class from a previous exercise and use the **ArrayList** class to create a list of *WordPairs*.

Example output:

```
=== Creating an English-Finnish dictionary (ok ends) ===
Enter an English word: autumn
Enter a Finnish word: syksy

Enter an English word: winter
Enter a Finnish word: talvi

Enter an English word: ok

=== English-Finnish translation service (quit ends) ===
Enter an English word: winter
talvi

Enter an English word: spring
Unknown word

Enter an English word: quit
Bye!
```

Objects Crew Program

Suppose that we want to create crews for boat cruises. Each crew can include several sailors.

INSTRUCTION

Write classes **Sailor** and **Crew** as below. These classes should work with the ready-made program class.

Sailor
- String name - String email
+ Sailor(String name, String email) + String getName() + String getEmail() + void setEmail(String email)

Crew
- ArrayList<Sailor> sailorList
+ Crew() + void addCrewMember(Sailor sailor) + String toString()

NOTE! Use the ready-made program class below to test your own classes. You should not modify the program class at all.

```java
public class ObjectsCrewProgram {

   public static void main(String[] args) {

       Sailor firstSailor = new Sailor("Frank", "frank@mail.com");
       Sailor secondSailor = new Sailor("Susan", "susan@mail.com");
       Sailor thirdSailor = new Sailor("John", "john@sailors.com");
       Sailor fourthSailor = new Sailor("Ann", "ann@sailors.com");

       Crew firstCrew = new Crew();
       Crew secondCrew = new Crew();

       firstCrew.addCrewMember(firstSailor);
       firstCrew.addCrewMember(secondSailor);
       firstCrew.addCrewMember(fourthSailor);

       secondCrew.addCrewMember(thirdSailor);
       secondCrew.addCrewMember(secondSailor);

       System.out.println("=== First crew ===\n" + firstCrew);
       System.out.println("=== Second crew ===\n" + secondCrew);

       secondSailor.setEmail("Susan@sailors.com");
       System.out.println("===Second crew ===\n" + secondCrew);
    }
}
```

Example output:

```
=== First crew ===
Frank (frank@mail.com)
Susan (susan@mail.com)
Ann (ann@sailors.com)

=== Second crew ===
John (john@sailors.com)
Susan (susan@mail.com)

=== Second crew ===
John (john@sailors.com)
Susan (Susan@sailors.com)
```

Objects Company Program

Write classes **Department** and **Employee** that work correctly with the ready-made program class **ObjectsCompanyProgram** below. You should not modify the program class at all.

```java
public class ObjectsCompanyProgram {

    public static void main(String[] args) {

        Department firstDepartment = new Department(10, "Sales");
        Department secondDepartment = new Department(20, "Music");

        Employee[] employeeArray = {
            new Employee("John Doe", "john@abc.com", firstDepartment),
            new Employee("Hank Marvin", "hank@abc.com", secondDepartment),
            new Employee("Joan Baez", "joan@abc.com", secondDepartment)};

        for (Employee employee : employeeArray) {
            System.out.println(employee);
        }

        System.out.println();

        employeeArray[0].setDepartment(secondDepartment);

        for (Employee employee : employeeArray) {
            System.out.println(employee);
        }
    }
}
```

The ***Department*** class has two fields (`number`, `name`), a parametric constructor, and getters/setters for the both fields.

The ***Employee*** class has three fields (`name`, `email`, `department`), a parametric constructor, getters/setters for all the fields, and the toString method.

Example output:

```
John Doe (john@abc.com), Sales department
Hank Marvin (hank@abc.com), Music department
Joan Baez (joan@abc.com), Music department

John Doe (john@abc.com), Music department
Hank Marvin (hank@abc.com), Music department
Joan Baez (joan@abc.com), Music department
```